

Using a Validation Model to Measure the Agility of Software Development in a Large Software Development Organization

Mikio Ikoma¹ Masayuki Ooshima¹

Takahiro Tanida¹ Michiko Oba¹ Sanshiro Sakai²

¹ Software Division, Hitachi, Ltd. ² Dept. of Computer Science, Shizuoka University

mikio.Ikoma.yh@hitachi.com masaki.ohshima.ex@hitachi.com

takahiro.tanida.fd@hitachi.com michiko.oba.cq@hitachi.com sakai@inf.shizuoka.ac.jp

Abstract

This paper provides a metric for evaluating the agility of software development projects and organizations. The metric is based on a validation model derived from the V&V model. In this metric, agility is defined as minimizing the time between the production of intermediate deliverables and the validation of those deliverables. The major distinguishing feature of this metric is that it is independent of any particular software development process model, such as agile software development methods or the waterfall model. Therefore, this metric can be used for software development organizations that have a wide variety of software development projects with varying kinds of development processes. This metric has been shown to be practical in large software development organizations through exhaustive use in more than 7,000 projects over the last 9 years.

1. Introduction

In the software industry, if two software organizations develop a commercial software product of the same functionality and same quality, the organization that can develop it at a lower cost over a shorter period has an advantage. Therefore, just as it is important for manufacturers in other industries, it also important for software development organizations to measure and improve productivity and the development period.

Since the 1980s, Japan's major software vendors have adopted a unified development process known as the "Japanese software factory" approach [1] [2]. Using this approach, the vendors have collected

extensive data on various aspects of software development and products (for example, relating to the cost, quantity, and quality of projects) and have used this data to improve their productivity.

The metric of productivity (the amount of production divided by cost) is an important metric even now. In recent years, however, software development organizations have had to face an increasingly competitive market and run projects under a variety of environment conditions. In this situation, the organizations confront the following issues:

- In addition to increasing the productivity of software development, organizations also need to improve efficiency and quality in response to reduced development periods and constantly changing conditions.
- While following many different development processes, organizations need to measure and leverage metrics that are independent of any specific development process model.

To resolve these issues, this paper uses the notion of software development agility, which we initially define as "the ability to quickly build functionality and quality into software at an early stage". The paper proposes a software development model called the Validation model, and a derived metric called the Agility metric that can measure software development agility. To demonstrate the effectiveness of the model and the metric, we will show how the Validation model and the agility metric were applied in a large development organization, and explain how the model and metric were used to improve the agility of that organization.

The Validation model is based on the V&V (verification & validation) model (Boehm [3]), which is important for establishing software quality, and

focuses on how a planned software item is validated during the software development process. The agility metric combines the Validation model and existing efficiency models. In particular, the metric references both a turnover metric (widely used in financial and manufacturing fields) and iterative and incremental development processes [4] (widely used in software development). However, the agility metric is not used just to shorten the development process or to enable use of specific lightweight software development processes; the metric is also used to achieve the goals of fast development and high quality software.

This paper shows that, by combining the V&V model and a turnover metric, the values of the Agile Manifesto [5] can be modeled, and not just for iteration. We think that many of the values of the Agile Manifesto correspond to the agility notion of “how quickly, and at how early a stage, planned software items can be validated”.

The Validation model and the agility metric can be used to evaluate and improve values of the Agile Manifesto independent of any specific software development process or specific practice. Therefore, large software development organizations, which usually use a variety of software development processes, can use the model and the metric. In this paper, we introduce our experience of applying the agility metric to several thousand software development projects in a large organization, which has about 3,000 software engineers, and report the effectiveness of the metric.

2. Issues of Productivity in Software Development

Shortening the development period and improving quality are important in product manufacturing in any industry, not just software. Productivity is measured as

“the amount of production divided by cost”. In the field of software development, increased efficiency is indicated by a lower cost for implementing the same functional or non-functional requirements. Therefore, the metric of productivity is also important for software development. However, the productivity metric alone cannot measure reductions in development time or increases in software quality, both of which are required for current software development organizations and projects.

First, we will describe productivity issues from the viewpoint of shortening the development period. Figure 1 shows Gantt charts of three software development projects that develop the same functionality. Both project A and B adopt a waterfall process model and project B divides the project into three concurrent subprojects. Project C adopts an iterative and incremental development process.

As shown in Figure 1, project C can release software earlier than the others and keeps less inventory during the development period. In addition, project C has financial advantages compared with project A and B, as shown by Denne [6]. However, if you just measure the productivity (the amount of production divided by cost), there is no difference among these three projects.

Project A = Project B = Project C

Next, if you use the development period instead of cost (that is, if you measure “amount of production divided by period”), the result is as follows:

Project A = Project C > Project B

However, from the management perspective, Project C is better than Project B and there are few differences between Project A and Project B.

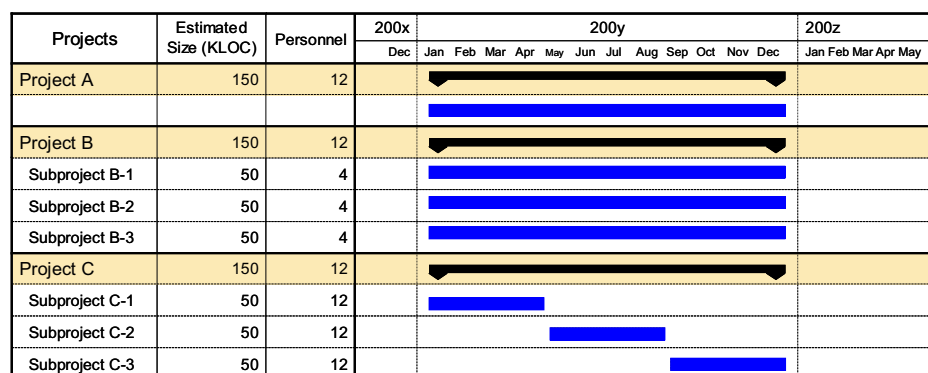


Figure 1: Examples of Gantt Charts for Software Development Projects of the Same Size

Therefore, productivity or “amount of production divided by period” cannot be used, and a metric that results in

$$\text{Project C} > \text{Project A} = \text{Project B}$$

is required for measuring the efficiency of software development organizations.

Furthermore, from the quality perspective, increasing productivity or shortening the development period should not affect the quality of the developed software product. To satisfy this paper's initial agility definition (“the ability to quickly, and at an early stage, build functionality and quality into software”), a metric is required that can evaluate how fast target software satisfies both functional and non-functional requirements of stakeholders.

3. Past Approaches and Issues

This section describes past approaches for measuring and improving the agility of software development, and issues with these approaches.

3.1. Issues with Metrics for Agile Development Methods

Four values described in the Agile Manifesto are important concepts for achieving the agility described in this paper. Therefore, metrics that can be applied to the agile development methods that conform to the Agile Manifesto can also be used to indirectly measure the agility of software development organizations.

For example, Qumer et al [7] showed the results of measuring six agile methods complying with the Agile Manifesto. Williams et al [8] proposed an evaluation framework called XP-EF to provide the means to check the fitness for each practice described in extreme programming (XP) [9]. Similar evaluation proposals can also be used for software development projects and their processes. Therefore, it is possible to measure the degree to which a project complies with the Agile Manifesto or XP, and to indirectly measure the agility of a project.

This approach may be used to measure the agility of software development organizations that employ a specific agile development method (ex. XP or Scrum [10]) in one or all their projects. However, as described

in Boehm [11] [12], in large software development organizations, many software development projects can adopt neither pure agile methods only, nor a pure waterfall model only. In short, adopting an agile software development method only is impractical. For example, there are many software development projects that have fixed requirements, for which heavyweight development processes similar to the waterfall model are used.

Therefore, to measure the agility of an entire software organization, it is difficult to use metrics that measure the fit with specific agile development methods. We need to use a metric that is independent of various software development process models.

3.2. Issues with Applying Financial or Hardware Manufacturing Metrics

The issues with specific agile development methods described in the previous section can be resolved by applying the financial indicator “turnover” and the hardware manufacturing indicator “cycle time” (used in the Just-In-Time (JIT) approach [13]) to software development organizations. That is, software items under development can be regarded as corresponding to assets in the financial indicator or corresponding to inventory in hardware manufacturing. Also, the amount of intermediate deliverables under development in relation to the completed deliverables indicates the turnover of software development (and cycle time is the reciprocal of the turnover).

In the examples in Figure 1, the average in-process inventories of Project A and Project B are both 150 KLOC in the year 200y and the amount of deliverables is 150 KLOC. The turnover of both projects is once per year. In the case of Project C, the average of the in-process inventories is 50 KLOC, the amount of deliverable is 150 KLOC, and the turnover of Project C is 3 times per year. Therefore one of the requirements for the agility metric is satisfied, namely:

$$\text{Project C} > \text{Project A} = \text{Project B}$$

Although this simple example compares projects having the same size, the metric can be used for many projects of various sizes. That is, the efficiency of a software development project or a software development organization is defined by the following formula:

$$E = V / U \quad (1)$$

Where:

- E is efficiency of a software development project or a software development organization.
- V is the entire amount of deliverables for a period.
- U is the average amount of intermediate deliverables for the period.

The unit of this value is turnover during a period for an organization. It represents how many cycles a software development project goes through during the period (that is, the reciprocal of the cycle time). Therefore, the value of the metric is independent of the unit of software size (SLOC, function points, etc.). A desirable feature of the metric is its broad utility: it is independent of any specific development process and can be used together with any software size metric.

The Software Division of Hitachi Ltd. has used the turnover metric to measure the efficiency of the software development organization since 1999. The results are shown in section 5. The same idea of applying a metric derived from cycle time in hardware manufacturing is proposed by Poppendieck [14] (applying lean manufacturing to software development) and Anderson [15] (applying the Theory of Constraints (TOC) to software development).

The turnover metric can be used to measure the efficiency of a software development organization. However, to measure agility, a metric is required that represents, not only efficiency, but also the quality of the software product and the values described in the Agile Manifesto. There are two issues that need to be resolved to use such a metric in real software development organizations.

- It is unclear what model is to be followed when measuring the turnover of software development. For example, what is inventory in software development? When does an item change from being inventory to being a final deliverable? And so on.
- It is unclear whether the metric is suitable or useful to the goals of software development projects or organizations. There is no point in measuring data for the metric if the only reason to do so is because you can measure it.

4. Agility Metric Based on the Validation Model

Section 4.1 proposes a validation model and metric that resolve the issues described in the preceding section. Section 4.2 shows the relationship between the proposed model and the values of the Agile Manifesto. Section 4.3 shows how software development organizations use the model and describes the metric. Section 4.4 notes some points to keep in mind when applying the validation model and metric.

4.1. Software Validation Model

This subsection describes how the agility of software development can be modeled using the V&V model, which is an important concept when considering the establishment of software quality.

The IEEE glossary [16] defines “verification” as follows:

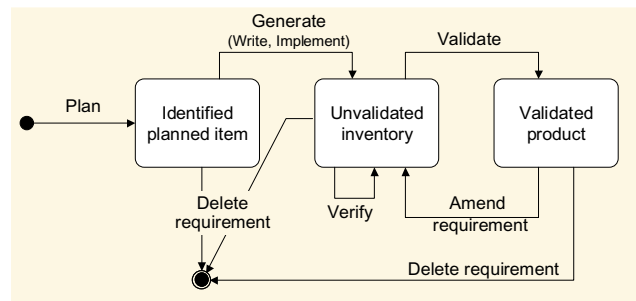
“The process of evaluating a system or component to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase.”

And the glossary defines “validation” as follows:

“The process of evaluating a system or component during or at the end of the development process to determine whether it satisfies specified requirements.”

Simple definitions are provided by Boehm [3]. That is, verification is “Are we building the product right?” and validation is “Are we building the right product?”

This paper focuses on validation because only validation can confirm the quality of the final software product. Based on this idea, this paper proposes a state transition model called the *Validation model* for any software items that can be validated (Figure 2).



**Figure 2. Validation Model
(Statechart for a Software Item)**

In this Validation model, any software items that can be validated enter an “identified planning state” at planning time. Then the items transition to the “unvalidated inventory state” when the software items start to be generated. Finally, validation of the deliverable changes the state to the “validated product state”. The Validation model places less significance on verification. Verification activities (such as reviewing development documents or inspecting source code) do not result in transitions to a following state, such as the “unvalidated inventory state”.

Note that development requirements are often changed during software development. Therefore, this model considers irregular state transitions, such as deleting a software item or transitioning from the “validated product state” to the “unvalidated inventory state”, to be changes in requirements.

The distinguishing feature of this model is that it introduces the state of an unvalidated intermediate product (or deliverable): that is, the inventory state.

This paper can now define software development agility as “the ability to minimize the period of the unvalidated inventory state” in the software validation model. The formula shown in 3.2 can now be interpreted using the software validation model. That is, the agility of a software development project or a software development organization is defined by the following formula:

$$A = V' / U' \quad (2)$$

Where:

- A is the agility of a software development project or a software development organization.
- V' is the entire amount of software items in the “validated product state” for a period.
- U' is the average amount of software items in which intermediate deliverables are in the “unvalidated inventory state” for the period.

To understand the agility metric intuitively, a simple example is provided below. Suppose two software projects develop software of the same size: one project uses the waterfall model, and one uses the agile development method. Figure 3 shows typical patterns of using verification and validation.

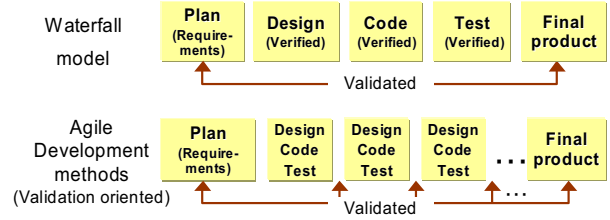


Figure 3. Typical Patterns in the V&V Model

In the waterfall model, during the development process, source code cannot be validated to check whether it satisfies project requirements, and verification is the only means of quality assurance. On the other hand, agile development methods consider that verification activities in intermediate development processes are unimportant and values the working software, which can be repeatedly validated against requirements during the development process.

Next, Figures 4 and 5 show the state transition of software items in the waterfall model and agile development methods.

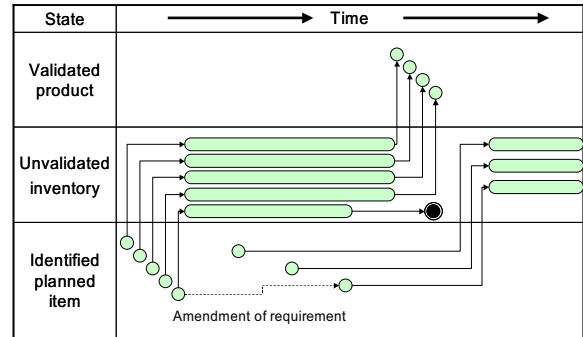


Figure 4. Typical State Transitions of Software Items When the Waterfall Model Is Used

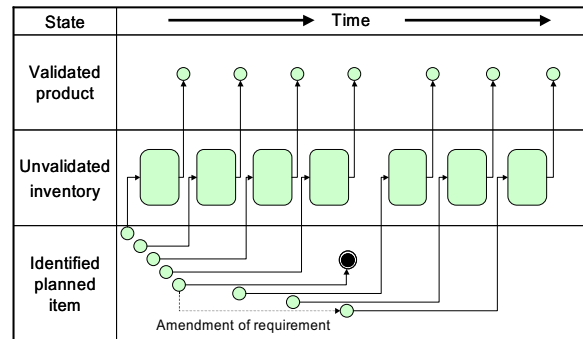


Figure 5. Typical State Transitions of Software Items When Agile Development Methods Are Used

As is obvious from these figures, the period of each software item in the inventory state is long when the waterfall model is used and short when agile development methods are used. (Note that this comparison focuses on the lengths of periods. It is possible that the agile development methods might be more costly.)

Figure 6 describes the transition of the amount of unvalidated inventory (that is, intermediate deliverables) during the development period.

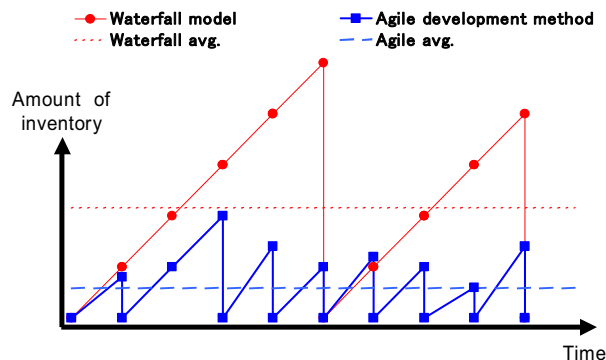


Figure 6. Transitions of Inventory (Intermediate Deliverables) In Different Process Models

In the waterfall model, inventory (such as unvalidated source code) accumulates until the end of the development period and is validated in the final step; then the inventory (intermediate deliverables) is cleaned up and the final deliverable is generated. Therefore the average number of inventory items during the whole process is large. In agile development methods, the average number of inventory items is small because there are repeated validations during the development period.

Therefore, if two projects develop the same size software, the project that has the smaller average of unvalidated inventory is more agile than the other.

4.2. Relationship Between the Validation Model and the Agile Manifesto

The Validation model, which is mainly derived from the V&V model, also corresponds to four values set in the Agile Manifesto. The following gives the four values and describes how the Validation model interprets them.

• Individuals and interactions over processes and tools:

Validation, which requires interactions with individuals to validate software, is more important than verification, which can be performed by tools or during intermediate processes.

• Working software over comprehensive documentation:

Although verification can be performed on intermediate deliverables such as documentation, final validation can be performed on working software only.

• Customer collaboration over contract negotiation:

Validation of customer satisfaction cannot be done with a contract, but requires collaboration with the customer.

• Responding to change over following a plan:

Validation must not be done according to a predefined plan but must match current changing requirements.

The proposed Validation model does not depend on the practices recommended by the Agile Manifesto but corresponds to basic values of the manifesto. Therefore, the model can be independent of specific development methods and it conforms to the spirit of the manifesto.

4.3. Process of Applying the Validation Model

This section describes how to apply the metric to large software development organizations that have many software development projects according to GQM (Goal Question Metric) by Basili [17].

Goal:

Clarify the agility-related business goals of the organization. The agility goals can be changed to match the particular characteristics of an organization. The goals might be as follows:

- How to increase the speed of turnover in software development projects (that is, apply software development projects themselves to the software items described in 4.1).
- How to shorten the time to define requirements of functional or nonfunctional items to be developed.
- How to shorten the period from coding to shipment.
- How to reduce the period required to detect a failure, find the corresponding fault, fix it, and then validate the fix.

Question:

Investigate the current situation of the organization and select the software items that are directly related to the business agility goals. The following are examples of software items:

- The software development project itself
- Source code (size, function points, SLOC, etc.) that corresponds to identified functional requirements or non-functional requirements
- Failures that occurred during testing or in the field

Data for the selected software items must be collectable for all software development projects in the organization. Next, generate questions about how the software items can achieve the goals, using the model described in section 4.1.

Metric:

Make preliminary measurements for a limited number of projects using the turnover metric for the selected software items. Analyze the results and determine whether the business agility goals can be accomplished. If the results are consistent with the goals, expand collection of the metric to all projects in the organization.

4.4. Validation Model's Main Features and Points to Consider When Applying It

(1) Special Features of the Validation Model

- The metric is meaningful for all stakeholders because it focuses on validation.
- The model does not depend on specific process models or development methods.
- The model can be used not only for the whole organization but also for a single software development project, which has an individual goal to improve agility.
- Not only the size of the source code described in section 3.1, but also any software items that will be validated can be used in this model.
- The metric derived from this model can be compared with, and also merged with, results from many projects that use various process models.
- A large software development organization, which runs many software development projects in parallel, can measure its agility.

(2) Points to Consider When Applying the Validation Model

When selecting software items, it is important that the items can enter the “validated product state” shown in Figure 2; that is, the items must be subject to validation. “Subject to validation” means that stakeholders of the corresponding requirements can determine whether the selected software items are valid or invalid. Therefore, even in the case of working software that can be tested, if the software subject to testing is a program library that has no external interface, the library is outside the scope of this model. On the other hand, in a situation where working software is not available, if a document defines the requirements, the customer who issued the corresponding requirements can validate the document. Therefore such software items are within the scope of this model.

Also, selected software items must be controlled by some kind of configuration management means (with or without software tools) to enable the item to be identified and changes in the item's state to be managed.

5. Introduction of an Application Example

This section describes how the model described in the previous section was used to improve agility in a large software development organization.

5.1. Outline

The Software Division, Hitachi Ltd. is a large software development organization of about 3,000 software engineers, and develops commercial package software products. The Software Division was founded in 1969. Since the 1970s, the Software Division has adopted a “Japanese software factory” approach to quantify and manage software productivity and software reliability.

Until the 1990s, the Software Division used metrics (such as “size of products/cost” and “number of faults/size of products”) that are not related to the development period or process. However, since the 1990s, the Software Division has faced stiff competition in the marketplace. The Software Division introduced new metrics to enable measurements for the goal of higher speed, in addition to the conventional goals of higher productivity and higher reliability. The Software Division set up quantitative targets for production management and quality management corresponding to the goals. Table 1 shows how these

targets correspond to the development period or processes.

This section introduces an example of applying the agile metric (described in section 4) to production management of the organization.

Table 1: Priority Targets of Production Management and Reliability Management

	Production Management	Reliability Management
Since latter 1970s:	Productivity, observance of delivery time, etc.	No. of faults in development processes, No. of failures after delivery, etc.
Since latter 1990s:	(In addition to the above) Agility, normalization of productivity for various projects	(In addition to the above) Quantitative evaluations based on identifying what upper processes need to remove the faults

5.2. Results of Applying the Validation Model

The Software Division continues to measure the agility of the organization and of all software development projects. The results from more than 7,000 projects have been accumulated since Oct. 1999. The metric used in the Software Division complies with the model shown in section 4. It uses estimated lines of code as inventory and final lines of code as the final product.

This subsection summarizes the results of comparing two groups in the Software Division. One group (Group B) actively worked to improve agility of their software development and the other group (Group A) did not.

Table 2 outlines the groups and their characteristics. The measurement period was four years. Each group handled more than 1,000 projects. Improving agility was one of Group B's organization objectives. Many of the software development projects in the group adopted practices to improve agility, such as prototyping and concurrent QA and an iterative and incremental development process.

Table 2: Outline of Groups

	Group A	Group B
No. of projects	1,649	1,185
Average project size	13.7 KLOC	26.8 KLOC
Common metrics	Software item: Lines of code Metric: Turnover of 6 months	
Measurement period	4 years (same period)	
Major development process model	Waterfall model	Waterfall model, Incremental model
Target market	Matured	Growing
Major software engineering methods	Automatic testing, reverse engineering, formal verification	Prototyping, concurrent QA, component based development

5.3. Summary of Results

Figure 7 shows the results of comparing the agilities of the two groups. Figure 8 shows the results of comparing the productivity of the two groups. In both Figure 7 and 8, the x-axis represents time (half year units), and the y-axis represents relative value when the results of the first time unit of Group B is set to 1. According to the results shown in Figure 7, applying the software development methods for improving agility doubled the agility of group B. On the other hand, no distinguishable change can be detected in the agility of Group A.

If you compare the results of productivity in Figure 8, you will see that the change in agility is not always the same as the change in productivity. In year 2H and 3H, a drastic drop in productivity was detected; however, agility remained at the same level. Group B doubled agility in four years, and achieved an approximate 30% increase in productivity.

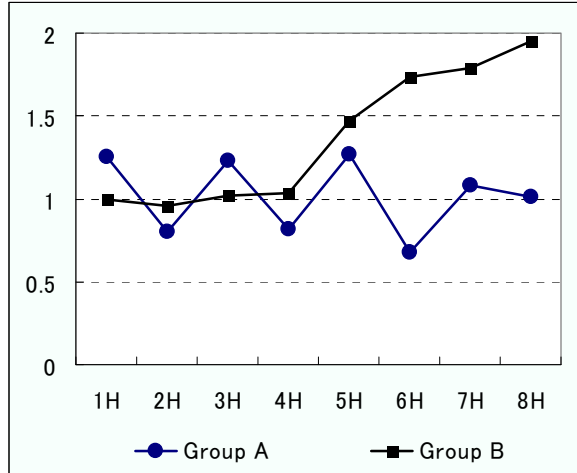


Figure 7. Changes in Agility of the Two Groups

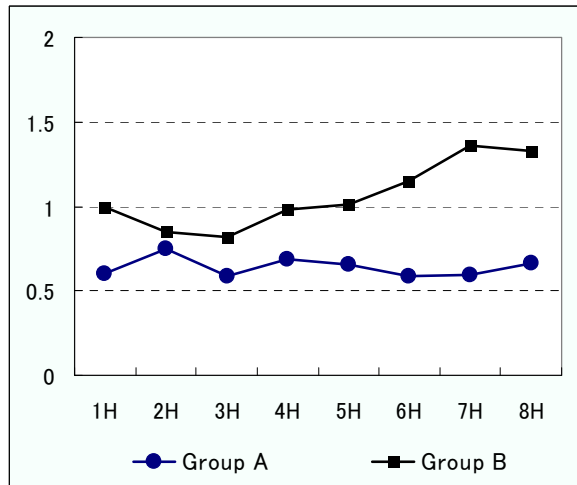


Figure 8. Changes in Productivity of the Two Groups

As both ROI and asset turnover are required in the financial world, we show that both productivity and agility are necessary in the field of software development.

Figure 9 shows the quality changes in Group B during the same period. The x-axis represents time (in half year units, during the same period as in Figure 7 and 8). The y-axis represents changes in the number of failures that occurred after release, divided by the total size of all development projects belonging to group B in that period. It plots numbers relative to the result of the first period, which is 1.

Although a remarkable improvement in quality cannot be detected from Figure 9, we can confirm that

improvement in agility does not damage the quality of the software.

In the example, agility improvement activities of Group B accurately reflect the results of the metric.

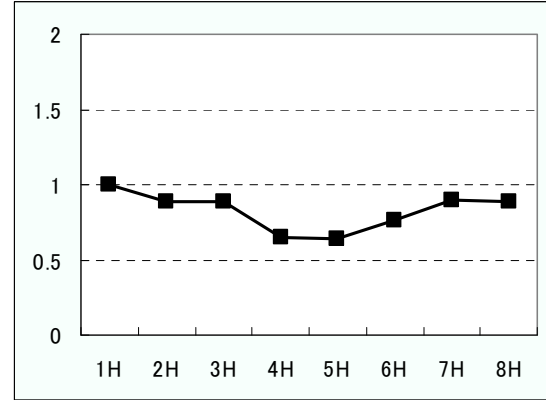


Figure 9. Changes in Field Failures of Group B

6. Conclusion and Future Tasks

This section summarizes the effectiveness of, and future tasks related to, the proposed agility metric for software development organizations and projects.

Effectiveness of the Agility Metric:

- This paper defines agility in software development as the minimization of the time from generation of intermediate deliverables to validation, and proposes a metric similar to turnover in finance or cycle time in hardware manufacturing, but corresponding to the characteristics of software development.
- The proposed metric is independent of any process model. The metric can be used in any existing process model, agile development method, or tailored process.
- The proposed metric can be used with conventional productivity measurements (such as “amount of production divided by cost”) and can be used to measure the agility of a software development project.
- The metric is also available for a large software development organization that runs many software development projects with various process models in parallel.

Future Tasks Related to the Metric:

- For practical use of the metric, an organization or project must have a mature development process. In particular, quantitative validation is required using a configuration management environment that supports

requirements management, quality management, and product management.

- Normalization of inventory and/or final products is required for development projects for which the measurement of inventory and/or final products is difficult, such as projects that involve outsourcing or component-based development.

In the future, we will continue to apply, evaluate, and improve the metric. In addition, we will investigate how to optimize the development process using the model described in this paper.

References

- [1] Cusumano, M. A.: Japan's Software Factories: A Challenge to U.S. Management: Oxford Univ Pr on Demand (1991)
- [2] Matsumoto, Y.: Japanese Perspectives in Software Engineering, pp303 Addison-Wesley(1989)
- [3] Boehm, B. W.: Verifying and Validating Software Requirements and Design Specifications, IEEE Software, Volume 1, Issue 1 (January 1984)
- [4] Craig Larman, Victor R. Basili: Iterative and Incremental Development: A Brief History: IEEE Computer 36 (6): pp.47–56: (June 2003)
- [5] Agile Manifesto. Manifesto for Agile Software Development, <http://agilemanifesto.org/>
- [6] Denne, M., Cleland-Huang, J.: The Incremental Funding Method, A Data Driven Approach to Software Development: IEEE Software, (May/June, 2004).
- [7] Qumer, A. et al: An evaluation of the degree of agility in six agile methods and its applicability for method engineering, Information and Software Technology 50 (2008) pp280-295
- [8] Williams, L. et al: Extreme Programming Evaluation Framework for Object-Oriented Languages Version 1.4, ftp://ftp.ncsu.edu/pub/unity/lockers/ftp/csc_anon/tech/2004/TR-2004-18.pdf, 2004
- [9] Kent Beck: Extreme Programming Explained, Embrace Change: Addison Wesley, 1999.
- [10] Schwaber, Ken: Agile Project Management with Scrum: Microsoft Press(2004)
- [11] Boehm, B. W.: Get Ready for Agile Methods, with Care: IEEE Computer, January 2002 (Vol. 35, No. 1) pp. 64-69
- [12] Boehm, B. W. et al: Balancing Agility and Discipline: A Guide for the Perplexed, Addison-Wesley(2004)
- [13] Ohno, T.: Toyota Production System: Beyond Large-Scale Production, Productivity Pr (1988)
- [14] Poppendieck, M. et al: Lean Software Development: An Agile Toolkit, Addison-Wesley(2003)
- [15] Anderson, D. J.: Agile Management for Software Engineering: Applying the Theory of Constraints for Business Results, Prentice Hall PTR (2003)
- [16] IEEE Std 610.12-1990, IEEE Standard Glossary of Software Engineering Terminology, 1990
- [17] Basili, V. et al: Using Measurement to Build Core Competencies in Software, <http://www.cs.umd.edu/~basili/publications/technical/T87.pdf> , (2005).